Contents

1 Lecture 1 **2**

1.1 Prediction From Expert Advice 2

2 Lecture 2 **5**

2.1 Relationship to Other ML Problems 5

2.2 Exploiting Structure 6

3 Lecture 3 **7**

3.1 Mathematical Optimization 7

3.2 Gradient Descent 7

4 Lecture 4 **10**

4.1 Online Portfolio Selection 10

4.2 Online Gradient Descent 10

4.3 Beating Gradient Descent 11

5 Lecture 5 **13**

6 Lecture 6 **14**

6.1 Statistical Learning Theory 14

1 Lecture 1

When one develops the theory of machine learning, it's a good idea to follow the following order:

$$\text{Problem} \longrightarrow \text{Theoretical Model} \longrightarrow \text{Algorithmic Method} \longrightarrow \text{Theoretical Guarantee}$$

Example 1.1
Here are some **problems** you may start with.

1. Prediction from expert advice
2. Email spam classification
3. Routing packets in a network
4. Portfolio selection
5. Robot control
6. Sequence prediction

1.1 Prediction From Expert Advice

In this setup, we perhaps have a decision between two options A and B. I have N experts giving advice on which option to pick. How do we choose the better option? This kind of setup doesn't make much sense with a one-shot position; we have no information on whether the experts give good advice.

How about the multi-shot setting? How can we hope to obtain the smallest number of mistakes? In some sense we wish to identify those experts that are "good." Well, if all the experts are bad, then we can't really hope to do better than them. Instead, a good model is to have the number of mistakes approach the best expert in hindsight.

Let's try a simple strategy to find this best expert. One simple strategy is to pick the expert that has been the best so far. However, one can see quickly that this does not work. For instance, suppose the predictions are as follows (T for true, F for false)

1	2
F	T
T	F
F	T
T	F

Then it's clear that we will alternate between the experts and never get it right! But the best expert in hindsight is right 50% of the time. The following **weighted majority** algorithm does better.

Algorithm 1.2 (Littlestone/Warmuth '89)
We will assign a weight to each experts i at each time step t, call it $W_t(i)$. Initially, for all i, $W_1(i) = 1$. To make a decision at time t, we pick the decision D that maximizes

$$\sum_{\text{expert } i \text{ makes prediction } D \text{ at time } t} W_t(i)$$

Every time step, we update the weight as follows. Take some parameter $\epsilon \in (0, 1)$.

$$W_{t+1}(i) = \begin{cases} W_t(i) & \text{if } i \text{ was right} \\ W_t(i) \cdot (1-\epsilon) & \text{otherwise} \end{cases}$$

Theorem 1.3
The number of mistakes made by the weighted majority,

$$M \leqslant 2(1+\epsilon)R + \frac{\log N}{\epsilon}$$

where R is the number of mistakes made by the best expert.

Proof
We will use a potential function argument to prove this. The potential function will be $\phi_t = \sum_{i=1}^{N} W_t(i)$, the total sum of the weights. It's clear that $\phi_1 = N$ and $\phi_{t+1} \leqslant \phi_t$. In fact, if the algorithm errored, then at least half of the weight was on the wrong choice, so therefore we decrease at least half the weight. Thus

$$\phi_{t+1} \leqslant \frac{1}{2}\phi_t + \frac{1}{2}\phi_t(1-\epsilon)$$

Which means in fact a strict decrease of the weights, wherein $\phi_{t+1} \leqslant \phi_t\left(1 - \frac{\epsilon}{2}\right)$. Thus, $\phi_T \leqslant \phi_1\left(1-\frac{\epsilon}{2}\right)^M$. Therefore, the highest weight is at most $N\left(1-\frac{\epsilon}{2}\right)^M$. The highest weight is also $(1-\epsilon)^R$. We thus have

$$(1-\epsilon)^R \leqslant N\left(1 - \frac{\epsilon}{2}\right)^M \implies M \leqslant 2(1+\epsilon)R + \frac{\log N}{\epsilon}$$

by using $-x + x^2 \leqslant \log(1-x) \leqslant -x$ for $0 < x < 1/2$.

In fact, this algorithm is optimal for deterministic algorithms (as $\epsilon \to 0$). The adversarial case is that the two experts always pick opposite things (fixed), and the one your algorithm picks is always wrong. Then, one of the experts, by pigeonhole, was right on at least half the days, but you were wrong every day. It turns out we can get rid of this problem if we use a distribution.

Algorithm 1.4 (Randomized Weighted Majority)
We make the decision D with probability

$$\frac{\sum_{\text{expert } i \text{ makes prediction } D \text{ at time } t} W_t(i)}{\sum_i W_t(i)}$$

The update and initial rules are the same.

Theorem 1.5
In the randomized weighted majority algorithm, M, follows

$$\mathbb{E}[M] \leqslant (1+\epsilon)R + \frac{\log N}{\epsilon}$$

Proof
The proof is very similar. The only difference is now we can reason about $\mathbb{E}[\phi_t]$. Let m_t be the indicator random variable on whether or not the algorithm made a mistake at time t. Let $m_t(i)$ be the indicator on whether expert i made a mistake (not random). Now, the new claim is $\phi_{t+1} \leqslant$

$\phi_t(1-\epsilon)^{\mathbb{E}[m_t]} \leqslant e^{-\epsilon\mathbb{E}[m_t]}$. It's not hard to see that this will immediately yield the result we want.

$$\begin{aligned}
\phi_{t+1} &= \sum_i W_t(i)(1-\epsilon m_t(i)) \\
&= \phi_t(1-\epsilon\sum_i p_t(i)m_t(i)) \\
&= \phi_t(1-\epsilon\mathbb{E}[m_t]) \leqslant \phi_t e^{-\epsilon\mathbb{E}[m_t]}
\end{aligned}$$

2 Lecture 2

To extend our results from before to the case where there are more than two possible predictions, we can think of "following" an expert instead of picking a specific option. We can model $\ell_t(i)$ as the "loss" (error measure) of expert i at time t. We can generalize the loss to model fractional errors, allowing $\ell_t(i) \in [0, \infty)$.

Algorithm 2.1 (Hedge Algorithm)
We again have weights $\{W_t(i)\}$ for the experts. We pick advice i time t with probability $\frac{W_t(i)}{\sum_j W_t(j)} =: p_t(i)$. We suffer (expected) loss $\mathbb{E}[L] = \sum_i p_t(i)\ell_t(i)$. Then, we update weights $W_{t+1}(i) = W_t(i)e^{-\epsilon \ell_t(i)}$.

Our yardstick is pretty similar.

Theorem 2.2
For all i,

$$\sum_{t=1}^{T} p_t^\top \ell_t \leqslant \sum_{t=1}^{T} \ell_t(i) + \epsilon \sum_{t=1}^{T} p_t^\top \ell_t^2 + \frac{\log N}{\epsilon}$$

Proof
We again define $\phi_t = \sum_i W_t(i)$.

$$\begin{aligned}
\phi_{t+1} &= \sum_{i=1}^{N} W_t(i)e^{-\epsilon \ell_t(i)} \\
&= \phi_t \sum_{i=1}^{N} p_t(i)e^{-\epsilon \ell_t(i)} \\
&\leqslant \phi_t \sum_{i=1}^{N} p_t(i)(1 - \epsilon \ell_t(i) + \epsilon^2 \ell_t^2(i)) \\
&\leqslant \phi_t e^{-\epsilon p_t^\top \ell_t + \epsilon^2 p_t^\top \ell_t^2}
\end{aligned}$$

Furthermore, the potential function is at least $\Phi_T \geqslant e^{-\epsilon \sum_t \ell_t(i)}$. Now $\Phi_T \leqslant Ne^{-\epsilon p_t^\top \ell_t + \epsilon^2 p_t^\top \ell_t^2}$. Taking a log and dividing by ϵ gives the result.

Observe that if $\ell_t(i) \in [0, 1]$, then the expected loss $\mathbb{E}[L] \leqslant \sum_t \ell_t(i) + \epsilon T + \frac{\log N}{\epsilon}$. If we pick $\epsilon = \sqrt{\frac{\log N}{T}}$, then

$$\frac{1}{T} \sum_t \left(p_t^\top \ell_t - \ell_t(i)\right) \leqslant 2\sqrt{\frac{\log N}{T}} \to 0$$

We can define regret as $R = \sum_t p_t^\top \ell_t - \ell_t(i)$.

2.1 Relationship to Other ML Problems

Consider the **online routing** problem. There is some graph $G = (V, E)$ with a source s and destination t. We wish to find the fastest path from s to t. The time taken to traverse each path constantly changes due to network traffic. We can reduce this back to the experts problems, where each path is an expert. There can be as many as $N!$ paths, so the regret will be $R \sim \sqrt{|E|\frac{\log N!}{T}} \sim \sqrt{\frac{N^3 \log N}{T}}$.

Consider the **spam classification** problem. Represent an email with a bag-of-words representation $v \in \{0, 1\}^d$. where $d = |\text{dictionary}|$. Then, a classifier is a vector $x \in \mathbb{R}^d$ with $|x| \leqslant 1$ and uses $\text{sign}(x^\top v)$ to

report spam or not spam. We can take "all possible" classifiers up to resolution ϵ where there are $\left(\frac{1}{\epsilon}\right)^d$ such classifiers. These will form our expert set, and then we can just run hedge.

However, both of these approaches miss out on the structure of the underlying problems. For instance, there are only $|E|$ numbers that determine the shortest path in the graph, but we seem to elide the fact that the paths largely share edges with each other.

2.2 Exploiting Structure

Structured decisions lead us to the world of mathematical optimization.

1. The set of all norm at most 1 linear classifiers is just the unit ball.
2. The set of all s-t paths in $G = (V, E)$ form a simplex. In particular, for some path P_{st}, define $x_e = 1$ if $e \in P_{st}$ and 0 otherwise. Then define set $K \subseteq R^{|E|}$,

$$K = \left\{ x \in \mathbb{R}^{|E|}, \sum_{i \in V} x_{si} = 1, \sum_{i \in V} x_{it} = 1, \forall j \sum_{i} x_{ij} = \sum_{k} x_{jk}, x_{ij} \in [0, 1] \right\}$$

   One may recognize this as the flow polytope. The corners of this polytope form paths from s to t.

3 Lecture 3

Consider the problem of Online TSP. Given $G = (V, E)$ over N nodes where there are $N!$ possible routes. If we do not mutate the instance, then if $P \neq NP$, then we would solve G in poly-time. In general, Hedge cannot be implemented faster than $\Omega(\#\text{ experts})$. The question to ask is then, what types of problems can I learn **efficiently**?

3.1 Mathematical Optimization

In mathematical optimization/programming, we have a set $K \subseteq \mathbb{R}^d$ and an objective function $f : K \to \mathbb{R}$. Our goal is to find a $x \in K$ such that $\forall y \in K\, f(x) \leqslant f(y)$, i.e. find the x that minimizes f.

Example 3.1

1. One example is linear regression. Given N points $\{(a_i, b_i) \ : i \in [N]\}$ we wish to find the line of best fit, i.e. $x \in \mathbb{R}^d$ that minimizes $\sum_{i=1}^{N} (a_i^\top x - b_i)^2$.
2. Another example is the shortest path problem. $K = \{0, 1\}^{|E|}$ and there's a function $W : E \to \mathbb{R}$ for a graph $G = (V, E)$. We wish to find $x \in K$ such that x is a path from s to t (two given nodes) and minimize $\sum_{e \in K} x_e W_e$.

When is MP solvable efficiently? When the MP is a convex program, there exist efficient algorithms to find solutions.

Definition 3.2
We say that K is convex if $\forall x, y \in K, \alpha \in [0, 1]$, $\alpha x + (1 - \alpha) y \in K$.

In other words, the line segment between x and y lies completely in the set.

Definition 3.3
f is convex over K if $\forall x, y \in K, \forall \alpha \in [0, 1]$,

$$f(\alpha x + (1 - \alpha) y) \leqslant \alpha f(x) + (1 - \alpha) f(y)$$

Theorem 3.4
An almost everywhere (real)-differentiable function f is convex if and only if

$$f(y) \geqslant f(x) + \nabla f(x)(y - x)$$

Theorem 3.5
Suppose a convex function has a local minimum at $x \in K$. That is, a small neighborhood around x, N, then $\forall y \in N$, $f(x) \leqslant f(y)$. Then $f(x)$ is also the global minimum.

In other words, any local minimum of a convex function is also a global minimum. Equivalently this means that $\nabla f(x) = 0$. In a constrained optimization problem, a point is only optimal if is gradient points outside the set.

3.2 Gradient Descent

To solve any convex optimization problem, there's an easy algorithm.

Algorithm 3.6 (Gradient Descent)
Let x_1 be arbitrarily chosen in K. For $t = 1, 2, \ldots$ we update

$$x_{t+1} = \Pi_k[x_t - \eta \nabla f(x_t)]$$

where $\Pi_k[x] = \operatorname{argmin}_{y \in K}\{|x - y|\}$. $\eta \geqslant 0$ is a real parameter called the learning rate or step size.

Proof 3.7
We will show that when K is closed, convex, and bounded, the above will find close to the optimum. In particular, if $\lambda \sim \frac{1}{\sqrt{T}}$, then

$$f\left(\frac{1}{T}\sum_{t=1}^{T} x_t\right) \leqslant f(x^*) + O\left(\frac{1}{\sqrt{T}}\right)$$

Let D be the diameter of K, $\max_{x,y \in K} |x - y|$. Let G be an upper bound on $|\nabla f(x)| \leqslant G$ for $x \in K$. This means that the Lipshitz constant of $f \leqslant G$, since

$$f(x) - f(y) \leqslant \nabla f(x)(y - x) \leqslant |\nabla f(x)||y - x| \leqslant G|x - y|.$$

Also due to projection, $|\pi_k[y] - x| \leqslant |y - x|$ for all $x \in K$.
Thus,

$$\begin{aligned}
|x_{t+1} - x^*|^2 &\leqslant |y_{t+1} - x^*|^2 \\
&= |x_t - \eta \nabla f(x) - x^*|^2 \\
&= |x_t - x^*|^2 - 2\eta \nabla f(x)(x_t - x^*) + \eta^2 |\nabla f(x)|^2
\end{aligned}$$

By convexity,

$$\begin{aligned}
f(x_t) - f(x^*) &\leqslant \nabla f(x_t)(x_t - x^*) \\
&\leqslant \frac{1}{2\eta}[|x_t - x^*|^2 - |x_{t+1} - x^*|^2] + \frac{\eta}{2}|\nabla f(x_t)|^2 \\
\frac{1}{T}\sum_{t=1}^{T}(f(x_t) - f(x^*)) &\leqslant \frac{1}{2\eta T}[|x_1 - x^*|^2 - |x_{T+1} - x^*|^2] + \frac{\eta}{2T} \cdot G^2 T \\
\frac{1}{T}\sum_{t=1}^{T} f(x_t) &\leqslant f(x^*) + \frac{D^2}{2\eta T} + \frac{1}{2}\eta G^2 \leqslant \frac{4GD}{\sqrt{T}} + f(x^*)
\end{aligned}$$

where in the last step we use $\eta = \frac{D}{\sqrt{TG}}$.

Let's return to our applications.

Example 3.8
1. Linear regression. We wish to solve

$$\min_{|x|_2 \leqslant 1} \frac{1}{n}\sum_{i=1}^{n} a_i^\top x - b_i)^2$$

The update is

$$x_{t+1} \leftarrow \Pi_K[x_t - \frac{1}{\sqrt{T}}\frac{2}{n}\sum_i (a_i^\top x - b_i)a_i]$$

Each iteration takes $O(nd)$ and we need to run $\Omega(1/\epsilon^2)$ iterations to guarantee error $O(\epsilon)$.

2. Suppose we want to find s-t shortest path. Let $K = \text{conv}(v \in \{0,1\}^E, v \text{ is a path})$. One can write capacity constraints.

$$K = \{x \in [0,1]^E, \sum_i x_{si} = 1 = \sum_i x_{it}, \forall j \sum_i x_{ij} = \sum_k x_{jk}, 0 \leqslant x_{ij} \leqslant 1\}$$

Then the objective function can be formulated as $\min_{x \in K} \sum_{i,j} x_{ij} W_{ij}$.

4 Lecture 4

4.1 Online Portfolio Selection

Consider d scores at each time t such that $x_t \in \Delta_d \subseteq \mathbb{R}^d$ where Δ_d is the probability simplex in d remensions. Consider return vector $r_t(i) = \frac{\text{price of } i \text{ on day } t+1}{\text{price of } i \text{ on day } t}$. Then, your total wealth is $W_{t+1} = W_t(x_t^\top r_t)$. The yardstick here should be the best constantly rebalancing portfolio in hindsight. For example, if I have the following:

Stock 1	2	$\frac{1}{4}$	2 ...
Stock 2	$\frac{1}{4}$	2	$\frac{1}{4}$ ...

Then it's clear that our best strategy would be $\begin{bmatrix} \frac{1}{2} & \frac{1}{2} \end{bmatrix}$. Taking logs, we have $\log W_{t+1} = \log W_t + \log\left(x_t^\top r_t\right)$. We want $\log W_T \to \sum_{t=1}^{T} \log\left(x_*^\top r_t\right)$ as $T \to \infty$. Thus, we can define

$$\text{Regret} = \sum_{t=1}^{T} \log\left(x_*^\top r_t\right) - \sum_{t=1}^{T} \log\left(x_t^\top r_t\right)$$

Applying Hedge and Gradient Descent kind of work, but they have some flaws.

4.2 Online Gradient Descent

Here is a more general framework to capture this problem.

Definition 4.1 (Online Convex Optimization)
Given $K \subseteq \mathbb{R}^d$ convex, $f_t : \mathbb{R}^d \to R$. We wish to choose $x_t \in K$ on each day. Our goal is to minimize regret, i.e.

$$\sum_t f_t(x_t) - \min_{x_* \in K} \sum_t f_t(x_*)$$

Algorithm 4.2 (Online Gradient Descent)
We run an update step on each day as

$$x_{t+1} \leftarrow \Pi_k[x_t - \eta \nabla f_t(x_t)]$$

It turns out this algorithm is pretty good.

Theorem 4.3

$$\text{Regret(OGD)} \leqslant 4GD\sqrt{T}$$

Proof
By projection,

$$|x_{t+1} - x^*|_2^2 \leqslant |x_t - \eta \nabla f_t(x_t) - x^*|_2^2$$

Expanding the LHS:

$$|x_t - \eta \nabla f_t(x_t) - x^*|_2^2 = |x_t - x^*|^2 - 2\eta \nabla f_t(x_t)(x_t - x^*) + \eta^2 |\nabla f_t(x_t)|^2$$
$$f_t(x_t) - f_t(x^*) \leqslant \nabla f_t(x_t)(x_t - x^*) \leqslant \frac{1}{2\eta}[(|x_t - x^*|^2 - |x_{t+1} - x^*|^2)] + \frac{\eta}{2} G^2$$
$$\text{Regret} \leqslant \frac{1}{2\eta}|x_1 - x^*|^2 + \frac{\eta}{2} G^2 T \leqslant GD\sqrt{T}$$

where in the last inequality we use $\eta = \frac{D}{G\sqrt{T}}$.

One can also try to apply this algorithm to the online routing problem, with the loss function $f_t(x) = \mathbb{E}[P \sim x]$ length of P. This is straightforward. One can also apply it to solving a regression problem, where the resulting algorithm is called Stochastic Gradient Descent.

Algorithm 4.4 (SGD For Regression)
Run OGD on randomly selected data point (a_t, b_t) and the loss is $(a_t^\top x - b_t)^2$ and return $\overline{x} = \frac{1}{T}\sum_t x_t$.

Theorem 4.5
SGD For Regression finds

$$\mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n}(a_i^\top \overline{x} - b_i)^2\right] \leqslant \text{OPT} + O\left(\frac{1}{\sqrt{T}}\right)$$

Proof

$$\frac{1}{T}\sum_t (a_t^\top x_t - b_t)^2 \leqslant \text{OPT} + O\left(\frac{1}{\sqrt{T}}\right)$$
$$\mathbb{E}\left[\frac{1}{n}\sum_t (a_t^\top \overline{x} - b_t)^2\right] \leqslant \frac{1}{T}\sum_t \frac{1}{n}\sum_i (a_i^\top x_t - b_t)^2$$

Where the last claim is by convexity (Jensen's).

It turns out the theorem we had before was tight; we can prove a matching lower bound.

Theorem 4.6
For general OGD, Regret $= \Omega(\sqrt{T}GD)$.

Proof (Sketch)
Let $K = [-1, 1]$ and $f_t(x) = x$ with probability $1/2$ otherwise $f_t(x) = -x$. Then we will keep bouncing back and forth with high probability and so we will not be able to figure things out.

4.3 Beating Gradient Descent

By the mean value theorem, any twice differentiable function satisfies for $\zeta \in [x, y]$:

$$f(x) = f(y) + \nabla f(y)(x - y) + (x - y)^\top \nabla^2 f(\zeta)(x - y)$$

A function is convex if and only if $\nabla^2 f(\zeta) \succeq 0$.

Definition 4.7
f is **strongly convex** if $\nabla^2 f(x) \succeq \alpha I$ for $\alpha > 0$ (i.e. $\nabla^2 f(x) - \alpha I$ is PSD).

Definition 4.8
f is β-smooth if $\nabla^2 f(x) \preceq \beta I$.

5 Lecture 5

We proved some better bounds on online gradient descent when the function is strongly convex. We also showed that non-convex optimization is really hard.

6 Lecture 6

It turns out non-convex optimization to find local minima is doable as long as the function is β smooth, i.e. $\beta I \succeq \nabla^2 f(x) \succeq -\beta I$. Assume $|f(x)| \leqslant M$ on the objective function, but there are no constraints. We will use Gradient descent.

Theorem 6.1
If $\eta = \frac{1}{\beta}$, then $\frac{1}{T}\sum_{t=1}^{T} |\boldsymbol{\nabla} f(x_t)|^2 \leqslant \frac{4\beta M}{T}$

Thus, we converge to a place where the gradient is zero.

In practice, the gradient is very expensive to compute. So instead suppose we have access to a stochastic gradient oracle, where

$$\mathbb{E}\left[\hat{\nabla} f(x)\right] = \nabla f(x); \;\; \mathbb{E}\left[|\hat{\nabla} f(x)|^2\right] = \sigma^2$$

and we perform update $x_{t+1} \leftarrow x_t - \eta \hat{\nabla} f(x_t)$.

Theorem 6.2
Stochastic gradient descent achieves $\mathbb{E}\left[\frac{1}{T}\sum |\nabla f(x_t)|^2\right] \sim \frac{1}{\sqrt{T}}$ for $\eta_t \sim \frac{1}{\sqrt{t}}$.

6.1 Statistical Learning Theory

Let's focus on the problems that statistical learning theory tries to capture. The input is a set of data X and labels Y. Underlying nature is some "concept" $c : X \to Y$. We can only sample data points from a (usually unknown) distribution $\mathcal{D}$ over $X \times Y$. Our goal is as follows. We define some loss $\ell : Y \times Y \to \mathbb{R}$ and then we wish to minimize generalization error

$$\mathbb{E}_{(x,y)\sim\mathcal{D}}\left[\ell(\hat{y}, y)\right]$$

where $\hat{y}$ is given by the algorithm. With these things alone, we cannot define our theory; there is nothing stopping us from just sampling from the distribution a bunch of times and only classifying things we've seen, allowing ourselves to be wrong on every other input; this is an issue called overfitting. We also need a hypothesis class $\mathcal{H} \subseteq |Y|^{|X|}$ of what kinds of functions we are allowed to have to approximate our concept. So now, the question we can ask is: which $\mathcal{H}$ are learnable?

Definition 6.3 (PAC Learning)
A learning problem $(X, Y, \mathcal{D}, \ell, \mathcal{H})$ is **statistically learnable** if there exists an algorithm for every $\epsilon, \delta > 0$ where the algorithm sees $m(1/\epsilon, \log 1/\delta)$ examples and produces $h \in \mathcal{H}$ such that $\mathbb{E}\left[[]\, \ell(h(x), y)\right] < \epsilon$ with probability at least $1 - \delta$.
m is called the **sample complexity** of the learning problem.

We can distinguish between two different assumptions on the hypothesis class. One is **proper learning**, which means $c \in \mathcal{H}$ and the other **agnostic learning**, which means $c \notin \mathcal{H}$, where we instead measure error with respect to closest member of the class (similar to regret).